

САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ, МЕХАНИКИ И ОПТИКИ
ФАКУЛЬТЕТ ПРИКЛАДНОЙ ИНФОРМАТИКИ

Отчет по лабораторной работе №2
по дисциплине «Методы оптимизации»
Тема: «Метод ломаных»

Выполнила:

Левчук С.А.

Группа: К3341

Поток: МетОпт 1.2

Санкт-Петербург

2026 г.

Постановка задачи

Задание на лабораторную работу №2 «Метод ломаных»

Разработайте программу на языке Python или подобном (C++, Java...), которая будет решать задачу поиска глобального экстремума для заданной функции на заданном отрезке.

На вход программа принимает строку одномерной функции, например $f(x) = x + \sin(3.14159 \cdot x)$ (она заведомо должна являться липшицевой), координаты концов отрезка (вещественные числа), точность вычисления ϵ (например, 0,01). На выходе она выполняет визуализацию графика исходной функции, вспомогательных функций (по возможности итоговой ломаной, на которой достигнута требуемая точность), приближенное значение аргумента и минимального значения функции, число пробовавшихся итераций, потраченной время.

Программа должна быть либо в Jupiter Notebook или в репозитории. Должно быть продемонстрирована работа программы на тестовой функции (любой одномерный сред двумерной функции): функция Растригина, функция Экли и др., но с наличием не менее двух локальных минимумов на заданном отрезке.

Результаты расчетов должны отражаться в виде визуализации (функция, ломаная линия или вспомогательные функции, указание места найденного приближенного значения аргумента и соответствующей точки минимума), приближенные значения аргумента и функции, число итераций, затраченное время для расчетов.

Демонстрация должна быть отражена в PDF файле, который вместе ссылкой на файл с кодом, с файл визуализация и(или) файл Jupiter Notebook, размещается в Moodle в ответ на это задание.

Программная реализация

Общая структура

Программа на Python, состоит из нескольких функций: `g`, `p`, `find_min_p`, `evaluate_l`, `pyavsky`, `choose_function`, `make_function`, `input_validation`, `print_result`, `plot_result`, `main`. Реализует метод ломаных (Пиявского).

`g` - вспомогательная функция

Строит “галочку” от точки u с наклоном $\pm L$. Принимает уже вычисленное значение W_u , чтобы не пересчитывать функцию.

```
def g(x, u, W_u, L):  
    """ вспомогательная функция  $g(x, u) = W(u) - L * |x - u|$  """  
    return W_u - L * abs(x - u)
```

`p` - ломаная

Ломаная $p_n(x)$ как максимум из всех галочек в точках `u_list`.

```
def p(x, u_list, W_list, L):  
    """ ломаная  $p_n(x) = \max_i g(x, u_i)$  """  
    return max(g(x, u_list[i], W_list[i], L) for i in range(len(u_list)))
```

`find_min_p` - поиск минимума ломаной

Ищет $u_n = \operatorname{argmin} p(x)$ по нижним вершинам: концы отрезка и точки пересечения соседних галочек.

```
def find_min_p(u_list, W_list, L):  
    """ поиск  $u_n = \min p(x)$  среди нижних вершин ломаной """  
    best_u, best_val = None, float('inf')  
  
    # кандидаты - концы отрезка и точки пересечения соседних  $g_i$   
    candidates = [u_list[0], u_list[-1]]  
  
    for i in range(len(u_list) - 1):
```

```

        ui, uil = u_list[i], u_list[i + 1]

        Wi, Wil = W_list[i], W_list[i + 1]

        # формула со слайда:  $u_{cross} = (W(u_q) - W(u_p)) / (2L) + (u_q + u_p) / 2$ 
        u_cross = (Wi - Wil) / (2 * L) + (ui + uil) / 2

        candidates.append(u_cross)

    for u in candidates:

        val = p(u, u_list, W_list, L)

        if val < best_val:

            best_val = val

            best_u = u

    return best_u, best_val

```

evaluate_l - оценка константы Липшица

Численная оценка через максимум модуля производной с запасом 1.2.

```

def evaluate_l(W, a, b, n=5000, k=1.2):

    """ оценка константы Липшица """

    xs = np.linspace(a, b, n)          # x_i

    ys = np.array([W(x) for x in xs])  # f(x_i)

    slopes = np.abs(np.diff(ys) / np.diff(xs)) # |f(x_{i+1}) - f(x_i)| / |x_{i+1} - x_i|

    return float(np.max(slopes)) * k   # max(...) * k

```

pyavsky - основной цикл

На каждой итерации: находим минимум ломаной, вычисляем функцию там, добавляем точку, проверяем условие остановки.

```

def pyavsky(W, a, b, eps, L, max_iter=10000):

```

```

""" метод Пиявского для поиска глобального минимума на [a, b] """

t_start = time.time()

# нулевая итерация: u_0 = a, u_1 = b
u_list = [a, b]
W_list = [W(a), W(b)]

iterations = 0
while iterations < max_iter:
    iterations += 1

    # u_n = min p_{n-1}(x), p_val - значение ломаной в этой точке
    u_new, p_val = find_min_p(u_list, W_list, L)
    W_new = W(u_new)

    # добавляем новую точку в отсортированный список
    idx = np.searchsorted(u_list, u_new)
    u_list.insert(idx, u_new)
    W_list.insert(idx, W_new)

    # лекционный критерий: W_new - p_val < eps
    if W_new - p_val < eps:
        break

elapsed = time.time() - t_start
best_idx = int(np.argmin(W_list))

return {
    'u_list': u_list,
    'W_list': W_list,
    'u_min': u_list[best_idx],

```

```

        'W_min': W_list[best_idx],

        'iterations': iterations,

        'time': elapsed

    }

```

choose_function - меню

Выводит список тестовых функций, читает выбор пользователя, возвращает функцию, отрезок и оцененную L.

make_function - строка в функцию

Превращает строку с выражением в вызываемую функцию через eval с ограниченным окружением.

```

def make_function(expr):

    """ превращает строку с выражением в функцию W(x) """

    code = compile(expr, "<expr>", "eval")

    def W(x):

        env = dict(SAFE_ENV)

        env["x"] = x

        return eval(code, env)

    return W

```

input_validation - валидация выражения

Запрашивает выражение, проверяет что оно вычисляется, при ошибке просит ввести его заново.

```

def input_validation():

    """ запрашивает выражение и проверяет, что оно вычисляется """

    while True:

        expr = input("f(x) = ").strip()

        try:

            W = make_function(expr)

            W(0.0)

```

```

        return W

    except Exception as e:

        print(f"ошибка в выражении: {e}, попробуйте ещё раз")

```

print_result и plot_result - вывод

Печатают u , $W(u)$, итерации, время. Строят график: функция, ломаная, галочки, вершины, минимум.

main - ввод данных

Запрашивает функцию из меню, ϵ , вызывает `pyavsky` и печатает результаты.

Примеры работы программы:

выберите тестовую функцию:

№	имя	$W(x)$	$[a, b]$
1	Растрингин	$10 + (x - 1)**2 - 10*\cos(2*\pi*x)$	$[-2.0, 3.0]$
2	Экли	$\cos(x) * \exp(-(x - \pi)**2)$	$[0.0, 10.0]$
3	Из задания	$x + \sin(3.14159*x)$	$[-2.0, 2.0]$
4	своя функция		

номер: 1

Оцененная $L = 80.7988$

$\epsilon = 0.001$

функция: Растрингин

отрезок: $[-2.0, 3.0]$

$L = 80.7988$, $\epsilon = 0.001$

Рисунок 1 – Выбор функции и ввод ϵ

```

=====
результаты:
  u*      = 0.9999948732
  W(u*)   = 0.0000000052
  итераций = 472
  время   = 6.4569 сек
=====

```

Рисунок 2 – Вывод результата

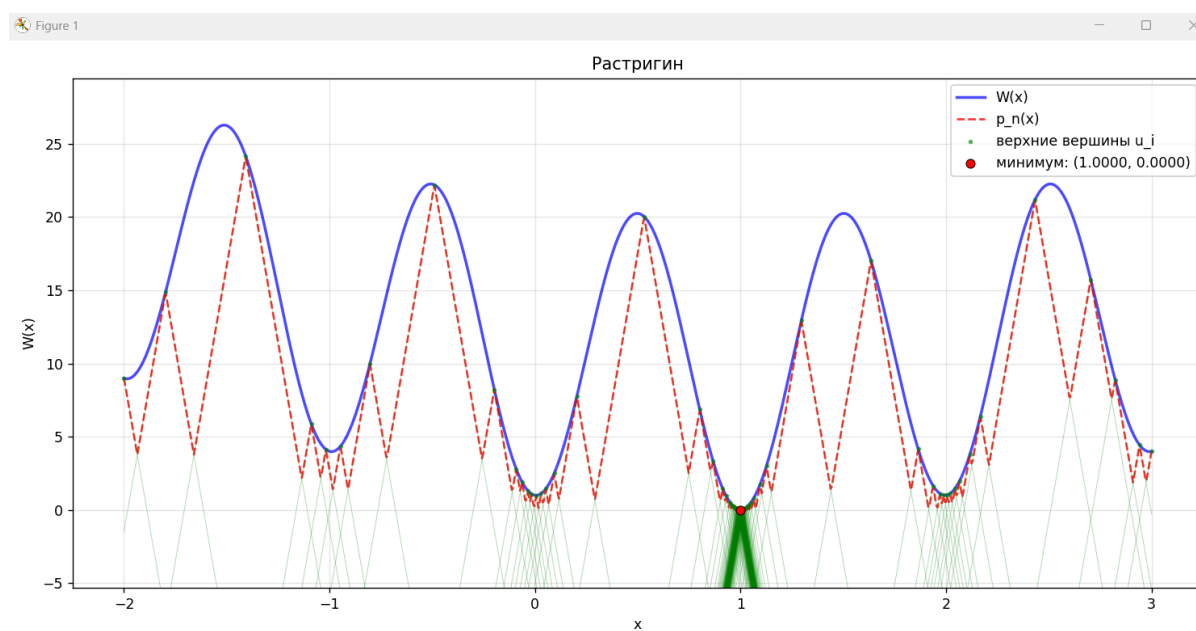


Рисунок 3 – Вывод графика

Выводы

Лабораторная работа оказалась довольно занимательной. В начале были небольшие трудности с тем, чтобы привыкнуть к методу, разобраться с формулами и нигде не запутаться. Но после аккуратного прохождения по алгоритму удалось справиться с этим и приступить к программной реализации.

Главной сложностью я бы назвала работу с константой Липшица. Сначала я задавала ее вручную, поэтому приходилось оценивать ее вручную. По невнимательности я путалась в вычислениях, и это приводило к некорректной работе. Но в дальнейшем я поменяла оценку константы на программную: добавила автоматическую оценку через производную с запасом 1.2 и все заработало корректно.

Еще одной трудностью было разобраться с производительностью. Изначально значения функции пересчитывались многократно, время работы было большим. После того, как я вынесла их в отдельный список, скорость выросл. Также была замечена ошибка в порядке проверки условия остановки, исправила ее на лекционный критерий, чтобы точка не терялась при остановке.

В целом работа была интересной, потому что мне нравится работать с функциями, наблюдать за их графиками.